

CAST: Testing the Attribution Stability of AI-Generated-Code Detectors Under Controlled Source Transformations

Anonymous Author(s)
Submission to ICSE 2027 NIER Track

Abstract— AI-generated-code detectors are evaluated almost exclusively on static snapshots, even though real code is routinely reformatted, renamed, and restructured after creation. We present CAST (Code Attribution Stability Testing), a controlled, distance-instrumented methodology that measures whether a detector’s human-versus-AI attribution survives such changes, and uses a canonicalization intervention to diagnose the source of any instability. Evaluating three independently reproducible detectors on 199 held-out Python and Java programs from the Droid corpus, we find qualitatively different stability profiles under identical transformations: DroidDetect-Base misclassifies 34.9% of previously correct Python samples and 24.1% of Java samples after reformatting alone, versus LLMSniffer’s stable behavior and DetectCodeGPT’s diffuse, family-independent instability. On pairs that remain non-identical after canonicalization – excluding those it trivially resolves – canonicalization still attenuates DroidDetect-Base’s formatting-induced score movement by 14.8–15.4 $\times$, but leaves DetectCodeGPT’s movement unreduced (and, on Java, larger), suggesting the two failure modes have different causes. Static accuracy alone therefore does not characterize detector reliability under routine code evolution.

Index Terms—AI-generated code detection, robustness testing, metamorphic-style evaluation, software evolution

I. INTRODUCTION

AI coding assistants now write a substantial share of the code developers review, edit, and ship. Detectors that attribute this code to a human or AI origin are typically evaluated the same way a classifier is: on a fixed test set, scored once, with accuracy as the verdict.

Motivation: That protocol quietly assumes the code being judged stays put. It does not. An IDE reformats a file on save, a refactor renames a variable, a reviewer rewrites a loop – none of it changes what the program does, yet any of it can change what a detector says about who wrote it. A detector whose verdict flips under a cosmetic edit is not measuring origin; it is measuring the snapshot. This is a routine, non-adversarial threat distinct from the adversarial setting the closest prior work targets: robustness studies vary how code is generated [9] or train against an attacker’s renaming strategy [6], [1], and separate work shows renaming can defeat code watermarks [7] and that code-model metamorphic testing treats renaming as a standard transformation [8]. None of this measures whether an already-trained AI-code detector’s verdict survives a non-

adversarial edit, or reports a stratified account of which edits matter and why.

Objective: This paper’s objective is to make attribution stability an explicit, measurable property of an AI-code detector, not an afterthought to accuracy, and to build the smallest instrument that can measure it. We introduce **CAST** (Code Attribution Stability Testing): given a program of known origin, CAST applies a validated transformation, measures the resulting change at the text, token, and AST level, and records how the detector’s score and decision move (Section II). Where movement is large, CAST adds a second, targeted step: canonicalizing both sides of the pair to test whether the instability is a presentation artifact or something deeper. Applied to three independently reproducible detectors over 199 held-out Python and Java programs [1], this approach exposes stability profiles that a single accuracy number cannot: one detector’s instability is almost entirely a formatting artifact that canonicalization removes; another’s is not.

This paper makes two contributions.

- **CAST**, a controlled, distance-instrumented diagnostic methodology for evaluating the attribution stability of already-trained AI-code detectors under routine, non-adversarial edits, including a canonicalization intervention that isolates presentation-driven instability from a broader failure mode.
- A held-out evaluation of three reproducible detectors over 199 programs, showing qualitatively different, transformation-specific stability profiles, together with a corpus-composition limitation that bounds this evaluation’s current reach.

The remainder of the paper is organized as follows. Section II defines CAST and the canonicalization intervention. Section III describes the dataset, transformations, detectors, and research questions. Section IV reports results; Section V discusses their implications. Section VI lists threats to validity, Section VII positions CAST against related work, Section VIII outlines future plans, and Section IX concludes.

II. CAST

A. Attribution stability

Attribution stability is the degree to which a detector’s output for a program is preserved under routine, non-adversarial edits – distinct from adversarial robustness, which studies a

Figure 1: CAST -- Controlled, distance-instrumented Attribution-Stability Testing

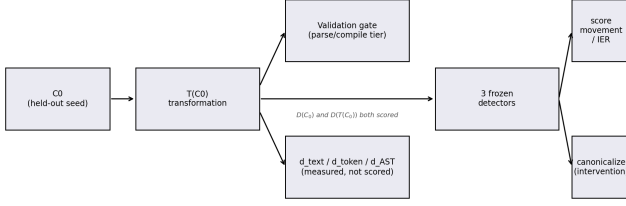


Fig. 1. The CAST framework.

worst-case attacker searching for an evasive rewrite. CAST’s transformations are fixed, deterministic operators applied uniformly across the corpus, not an adversarial search against any detector. CAST treats score movement and decision-flip rate as first-class observations, not a single post-transformation accuracy number.

B. Methodology

For a detector D , a transformation T , and a program C_0 of known provenance, CAST proceeds in five steps.

- 1) Apply T to obtain $T(C_0)$.
- 2) Validate that $T(C_0)$ is well-formed at or above C_0 ’s own syntactic-validity tier. Non-applicable and invalid outcomes are recorded, not discarded.
- 3) Measure d_{text} (character-level edit distance), d_{token} (edit distance over the language’s own lexical token stream, e.g., Python’s `tokenize` output including `INDENT/DEDENT`, or Java tree-sitter leaves), and d_{AST} (AST node-count change).
- 4) Score $D(C_0)$ and $D(T(C_0))$.
- 5) Aggregate, per detector, language, and family, the mean and median score movement $\Delta_D = D(T(C_0)) - D(C_0)$, the decision-flip rate, and the **Induced Error Rate**, defined as:

$$\text{IER} = \frac{\#\{\text{baseline-correct programs whose decision flips after } T\}}{\#\{\text{baseline-correct programs}\}} \quad (1)$$

IER is conditioned on the detector having been correct on C_0 . A flip on a program the detector already misclassified would otherwise conflate ordinary detector error with transformation-induced instability. Detectors also expose incompatible score semantics: a calibrated probability, an uncalibrated sigmoid output, or an unnormalized statistic. CAST therefore does not compare raw $|\Delta_D|$ across detectors. IER, a bounded proportion with a consistent interpretation regardless of score scale, is the primary cross-detector metric. Raw score movement is interpreted only within a given detector’s own distribution.

C. Canonicalization intervention

When a transformation family produces large score movement, CAST additionally scores $F(C_0)$ and $F(T(C_0))$ for a deterministic canonicalizer F that removes a specific, named category of variation (in this study: whitespace, blank lines, comments, and quote style, implemented by re-tokenizing the

program and rejoining its semantic tokens). A pair for which $F(C_0)=F(T(C_0))$ collapses to $\Delta=0$ by construction. This subset is reported separately from the residual pairs for which a genuine difference remains after canonicalization, so that the diagnostic result is not an artifact of the trivially-resolved cases. A substantial reduction in the residual subset’s score movement constitutes intervention evidence that the removed category of variation contributes to the observed instability. It does not constitute proof of a specific internal mechanism.

III. STUDY DESIGN

Dataset: This evaluation uses DroidCollection’s [1] held-out test split: 199 programs (100 Python, 99 Java), stratified across `HUMAN_GENERATED`, `MACHINE_GENERATED`, and `MACHINE_REFINED`.

Transformations: Three confirmatory families: formatting (`black/google-java-format`), lexical renaming of one identifier, and control-flow rewriting (one canonical loop-form rewrite), with applicability rate 0.616 and success rate 0.992 among applicable attempts. A fourth stage attempted two deeper families, local structural extraction and semantics-preserving rewriting, reported as a feasibility finding (Section IV-D), not a confirmatory one.

Validation: Python uses `compile()`-based validation. Java is tiered, given the corpus’s many function/class fragments: `STANDALONE_VALID` > `WRAPPER_VALID` (compiles inside a synthetic harness, used only as a validation instrument and never scored) > `STRUCTURAL_ONLY` > `PARSE_INVALID`.

Detectors: Three of five targeted detectors were independently reproducible from public artifacts. *LLMSniffer* [2] (GraphCodeBERT [17] with a supervised-contrastive head) reaches 90.0% sanity-check accuracy on GPTSniffer’s Java test subset [3]. *DroidDetect-Base* (ModernBERT-base [16] with a projection layer and a four-class head) ships a checkpoint with no modeling code; we reconstructed its architecture and validated it against a disjoint dev-split subset, reaching 87.5% accuracy. *DetectCodeGPT* [4] is a zero-shot statistic from the DetectGPT [12]/NPR [13] family, computed from CodeLlama-7B [14] and CodeT5p-770M [15], with $K=50$ perturbations per a pre-registered convergence study. Its threshold, calibrated on an independent 40-sample set, reaches only 60.0% accuracy, so raw score movement is primary evidence and threshold-derived decisions secondary. *GPTSniffer* [3] and *CodeGPTSensor*+/+ [5] could not be integrated, as neither released a checkpoint; we did not train replacements, to avoid confounds distinct from evaluating a published detector.

RQs: This evaluation is structured around three research questions.

- **RQ1:** How stable are detector scores and decisions under formatting, lexical renaming, and control-flow rewriting?
- **RQ2:** Do detectors exhibit similar or detector-specific stability profiles under identical transformations?
- **RQ3:** Does canonical normalization attenuate transformation-associated instability, and does this differ across detectors?

TABLE I

INDUCED ERROR RATE BY DETECTOR, LANGUAGE, AND FAMILY (n = BASELINE-CORRECT DENOMINATOR; $\dagger$ = SECONDARY EVIDENCE).

Detector	Lang.	Family	IER (n)	95% CI
DroidDetect-Base	Py	formatting	34.9% (29/83)	[25.6,45.7]
DroidDetect-Base	Java	formatting	24.1% (13/54)	[14.6,36.9]
DroidDetect-Base	Py	lexical rename	4.1% (3/73)	[1.4,11.4]
DroidDetect-Base	Java	lexical rename	2.8% (2/71)	[0.8,9.7]
DroidDetect-Base	Py	control flow	3.8% (1/26)	[0.7,18.9]
DroidDetect-Base	Java	control flow	7.1% (1/14)	[1.3,31.5]
LLMSniffer	Java	formatting	8.3% (2/24)	[2.3,25.8]
LLMSniffer	Py	formatting	6.5% (4/62)	[2.5,15.4]
LLMSniffer	Py	lexical rename	1.9% (1/52)	[0.3,10.1]
LLMSniffer	Java	lexical rename	0.0% (0/35)	[0.0,9.9]
LLMSniffer	Py	control flow	0.0% (0/15)	[0.0,20.4]
LLMSniffer	Java	control flow	0.0% (0/4)	[0.0,49.0]
DetectCodeGPT $\dagger$	Py	control flow	50.0% (7/14)	[26.8,73.2]
DetectCodeGPT $\dagger$	Py	lexical rename	30.2% (13/43)	[18.6,45.1]
DetectCodeGPT $\dagger$	Py	formatting	25.0% (12/48)	[14.9,38.8]
DetectCodeGPT $\dagger$	Java	formatting	16.7% (6/36)	[7.9,31.9]
DetectCodeGPT $\dagger$	Java	lexical rename	16.7% (8/48)	[8.7,29.6]
DetectCodeGPT $\dagger$	Java	control flow	18.2% (2/11)	[5.1,47.7]

IV. RESULTS

A. RQ1/RQ2: detector-specific, transformation-specific profiles

Table I reports IER with baseline-correct denominators. DroidDetect-Base’s formatting IER exceeds its lexical-rename and control-flow IER by 3.4–9.2 $\times$ within the same language, with non-overlapping Wilson [18] 95% CIs in three of four same-language comparisons; the exception is Java control-flow, whose small $n=14$ produces a wide, overlapping interval. LLMSniffer’s IER stays below 8.3% throughout. DetectCodeGPT’s IER is elevated broadly (16.7–50.0%), peaking under control-flow rather than formatting – diffuse, not family-localized; suggestive given its calibration caveat and not yet checked against re-scoring noise (Section VI). Median absolute score movement is consistent: little except under formatting for DroidDetect-Base (median 0.0055, up to 0.14); uniformly small for LLMSniffer (median 0.0106); broad regardless of family for DetectCodeGPT (median 0.478).

Answer: Stability is detector- and transformation-specific, not uniform.

B. RQ3: canonicalization intervention

The canonicalization diagnostic was applied to all 158 formatting pairs (94 Python, 64 Java). Canonicalizing both sides collapsed **108/158 (68.4%)** to byte-identical input ($\Delta=0$ by construction); the remaining **50/158** retained a residual difference and were rescored. Table II reports both, isolating the diagnostic’s contribution from the trivially-resolved cases.

On the residual-only subset – which excludes pairs canonicalization trivially equalizes – DroidDetect-Base’s mean score movement still falls 14.8–15.4 $\times$, intervention evidence that presentation variation drives most of its instability. DetectCodeGPT shows no comparable reduction: its Python residual movement is essentially unchanged (1.0 $\times$), and its Java movement increases (0.7 $\times$, mean $|\Delta|$ 0.566 $\rightarrow$ 0.866), so its insta-

TABLE II

MEAN $|\Delta|$ BEFORE/AFTER CANONICALIZATION, BY LANGUAGE (NOT COMPARABLE ACROSS DETECTORS).

Detector	Lang.	Full set		Residual-only	
		before	after (red.)	before	after (red.)
LLMSniffer	Py (94/32)	0.027	0.003 (8.6 $\times$)	0.032	0.009 (3.5 $\times$)
LLMSniffer	Java (64/18)	0.025	0.002 (16.7 $\times$)	0.020	0.005 (3.7 $\times$)
DroidDetect-Base	Py (94/32)	0.315	0.009 (34.9$\times$)	0.394	0.027 (14.8$\times$)
DroidDetect-Base	Java (64/18)	0.161	0.003 (63.3$\times$)	0.139	0.009 (15.4$\times$)
DetectCodeGPT	Py (94/32)	0.687	0.280 (2.5 $\times$)	0.843	0.824 (1.0 $\times$)
DetectCodeGPT	Java (64/18)	0.645	0.244 (2.6 $\times$)	0.566	0.866 (0.7$\times$, increase)

Canonicalization intervention – within-detector before/after (each panel its own y-axis; scales are NOT comparable across detectors)

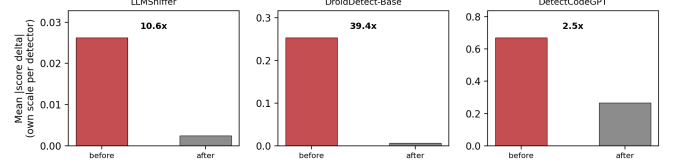


Fig. 2. Canonicalization reduction factor, per detector (not cross-comparable).

bility is not explained by the same presentation differences. LLMSniffer shows a smaller, real reduction on an already low baseline. Pooling both languages (Fig. 2) gives full-set factors of 10.6 $\times$ /39.4 $\times$ /2.5 $\times$; a ratio computed from rounded pooled means, rather than the underlying values, would overstate these by 2–3 $\times$, hence the language-level reporting here.

Cohen’s d_z [19] is reported in the accompanying artifact but not used as a headline statistic, since it standardizes by variance and can appear large even for a practically small movement (e.g., LLMSniffer, $d_z=1.39$ –1.49 despite mean movement below 0.025).

Answer: Canonicalization explains most of DroidDetect-Base’s residual instability; DetectCodeGPT’s is unreduced, and on Java, larger after canonicalization.

C. Corpus feasibility for deeper transformations

Local structural extraction and semantics-preserving rewriting were additionally attempted at two intensities. Of 796 attempted (language, family, intensity) combinations, only 37 (4.65%) were applicable, of which 35 validated successfully. Only 4 of those 35 received true differential-execution verification. The remaining 31 degraded to structural-only validation for lack of an unambiguous, stdin-free entry point. This outcome reflects Droid’s composition: a substantial share of its rows are function- or class-level fragments rather than complete executable programs, which bounds how densely this evaluation style currently extends to deeper transformations. The resulting per-detector cells ($n=1$ –12) are illustrative only and support no confirmatory claim.

V. DISCUSSION

Static accuracy does not characterize post-transformation reliability: detectors with non-trivial numbers of correct baseline calls (Table I’s n column) diverge sharply after a routine edit, though a low per-class baseline accuracy is itself a threat to

how an apparently “stable” IER should be read (Section VI). The two unstable detectors’ failure modes differ qualitatively, not merely in magnitude: DroidDetect-Base’s instability is narrow and substantially presentation-driven, whereas DetectCodeGPT’s is broad and unaffected by the same intervention. This motivates reporting stability under transformation alongside accuracy, and motivates canonicalization as a lightweight diagnostic that, for presentation-sensitive detectors, is also a candidate mitigation. CAST does not identify *why* a detector is presentation-sensitive or diffusely unstable; that requires inspecting internal representations or training data, outside this paper’s scope.

VI. THREATS TO VALIDITY

- **External:** The evaluation covers two languages, one corpus, and three reproducible detectors; two others could not be integrated for lack of a released checkpoint. The transformation operators represent common tooling edits, not sampled developer histories.
- **Construct:** Detector scores have incompatible scales (Section II-B), and DetectCodeGPT’s weak calibration (Section III) limits its threshold-derived results to secondary status. d_{text} , d_{token} , and d_{AST} are complementary but incomplete proxies for edit magnitude, and $\text{IER}/|\Delta|$ are not yet related to them directly. Neither a low IER nor a low per-class baseline accuracy has been checked against an identity-transformation (no-op, re-scored) noise floor, so an apparently stable profile is not yet distinguished from a near-constant prediction pattern.
- **Internal:** DroidDetect-Base’s architecture was reconstructed from its checkpoint absent public modeling code, and this evaluation does not yet report how its four-class output collapses to a human-versus-AI decision, so some counted flips could be between two machine-origin classes. The canonicalizer normalizes several presentation properties at once, so a positive result supports a claim about that combined category, not which specific property is responsible; detector accuracy on canonicalized inputs is not yet reported, so a movement reduction is not yet fully separated from the canonicalized input becoming an easier or out-of-distribution case.
- **Corpus:** Droid’s fragment-heavy composition limits the deeper transformations to 4.65% applicability and differential-execution verification to 4 of 35 successful instances; the corresponding per-detector cells are illustrative, not confirmatory.
- **Conclusion:** Some cells have small denominators, as low as $n=4$; Wilson 95% CIs are reported throughout for this reason. Cohen’s d_z is supplementary, directional-consistency evidence only.

VII. RELATED WORK

Table III positions CAST against the closest prior work on whether it edits code post-generation, whether that edit is routine rather than adversarial, whether it targets already-trained AI-code detectors, and whether it compares detectors

TABLE III
NOVELTY POSITIONING AGAINST THE CLOSEST PRIOR WORK.

Work	Post-gen. edit	Routine, non-adv.	AI-code detector	Cross-det.
Detector papers [3], [2], [4], [5]	partial	partial	✓	–
CodeGPTSensor+ [6], Droid [1]	✓	– (adversarial)	✓	partial
Pordanesh et al. [9]	–	–	✓	✓
RoPGen [10]	✓	✓	– (human authorship)	–
Non-adv. code edits [7], [8]	✓	✓	–	–
CAST (this work)	✓	✓	✓	✓

under identical edits. Detector papers [3], [2], [4], [5] report accuracy, occasionally under one perturbation, without stratifying transformations. CodeGPTSensor+ [6] shares the observation that renaming can obscure AI origin, but addresses it by training a detector against an adversary’s renaming, not by measuring an existing detector’s stability under a non-adversarial edit; Droid [1] similarly includes adversarial attack scenarios, not routine ones. Pordanesh et al. [9] vary training-set and prompting conditions, not the source code itself. RoPGen [10] applies style transformation *during training* to harden a *human*-authorship attributor; CAST transforms *at evaluation time*, against *already-trained*, unmodified AI-versus-human detectors. Renaming alone can erase a code watermark [7], and code-model metamorphic testing treats it as a standard, non-adversarial transformation [8], though neither targets AI-code attribution or compares detectors. Whitespace-based detection work [11] helps motivate why formatting sensitivity can be presentation-driven.

VIII. FUTURE PLANS

This evaluation will be extended in four directions. (1) An identity-transformation control, re-scoring C_0 with a fresh seed, to give each detector’s IER a measured noise floor. (2) A corpus of complete, executable programs with chained, multi-step transformations mined from real commit histories, to scale beyond the 4.65% applicability of function-level fragments (Section IV-D) and substitute a sampled distribution of edits for tooling-driven ones. (3) A component ablation of canonicalization (whitespace, comments, quote style scored separately) and an analysis relating IER and $|\Delta|$ to d_{text} , d_{token} , and d_{AST} , to test whether formatting’s outsized effect reflects edit size rather than edit type. (4) A detector explicitly trained against CAST’s transformation families, to test whether stability can be achieved by construction rather than only diagnosed.

IX. CONCLUSION

This paper presented CAST, a controlled, distance-instrumented methodology for testing attribution stability, applied to three reproducible detectors over 199 held-out Python and Java programs. The detectors show qualitatively different profiles: one shows formatting-specific sensitivity substantially attenuated by canonicalization; a second stays stable throughout; a third shows broad movement the intervention does not attenuate. Static accuracy alone is insufficient to characterize reliability under routine code evolution.

REFERENCES

- [1] D. Orel, I. Paul, I. Gurevych, P. Nakov, “Droid: A Resource Suite for AI-Generated Code Detection,” *EMNLP*, 2025.
- [2] M. L. Dihan, A. Muhtasim, “LLMSniffer: Detecting LLM-Generated Code via GraphCodeBERT and Supervised Contrastive Learning,” arXiv:2604.16058, 2026.
- [3] P. T. Nguyen, J. Di Rocco, C. Di Sipio, R. Rubel, D. Di Ruscio, M. Di Penta, “GPTSniffer: A CodeBERT-based classifier to detect source code written by ChatGPT,” *Journal of Systems and Software*, 2024.
- [4] Y. Shi, H. Zhang, C. Wan, X. Gu, “Between Lines of Code: Unraveling the Distinct Patterns of Machine and Human Programmers,” *ICSE*, 2025.
- [5] X. Xu, C. Ni, X. Guo, S. Liu, X. Wang, K. Liu, X. Yang, “Distinguishing LLM-generated from Human-written Code by Contrastive Learning,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 91, 2025.
- [6] X. Yin, X. Li, C. Ni, X. Xu, X. Yang, “Detecting LLM-generated Code with Subtle Modification by Adversarial Training,” arXiv:2507.13123, 2025.
- [7] T. Suresh, S. Ugare, G. Singh, S. Misailovic, “Is the Watermarking of LLM-Generated Code Robust?” arXiv:2403.17983, 2024.
- [8] A. Asgari, M. de Koning, P. Derakhshanfar, A. Panichella, “Metamorphic Testing of Deep Code Models: A Systematic Literature Review,” arXiv:2507.22610, 2025.
- [9] S. Pordanesh, S. Bukhari, B. Tan, L. De Carli, “Hiding in Plain Sight: On the Robustness of AI-Generated Code Detection,” *DIMVA*, 2025.
- [10] Z. Li, G. Q. Chen, C. Chen, Y. Zou, S. Xu, “RoPGen: Towards Robust Code Authorship Attribution via Automatic Coding Style Transformation,” *ICSE*, 2022.
- [11] S. M. H. Nirob, S. Ehsan, M. Rahman, S. Haque, “Whitespaces Don’t Lie: Feature-Driven and Embedding-Based Approaches for Detecting Machine-Generated Code,” arXiv:2601.19264, 2026.
- [12] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, C. Finn, “DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature,” arXiv:2301.11305, 2023.
- [13] J. Su, T. Y. Zhuo, D. Wang, P. Nakov, “DetectLLM: Leveraging Log Rank Information for Zero-Shot Detection of Machine-Generated Text,” *Findings of EMNLP*, 2023.
- [14] B. Rozière, J. Gehring, F. Gloeckle, et al., “Code Llama: Open Foundation Models for Code,” arXiv:2308.12950, 2023.
- [15] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, S. C. H. Hoi, “CodeT5+: Open Code Large Language Models for Code Understanding and Generation,” arXiv:2305.07922, 2023.
- [16] B. Warner, A. Chaffin, B. Clavié, et al., “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference,” arXiv:2412.13663, 2024.
- [17] D. Guo, S. Ren, S. Lu, et al., “GraphCodeBERT: Pre-training Code Representations with Data Flow,” arXiv:2009.08366, 2020.
- [18] E. B. Wilson, “Probable Inference, the Law of Succession, and Statistical Inference,” *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [19] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd ed. Lawrence Erlbaum Associates, 1988.